

Informe AA4 cultura digital

Nombre: Juan Piarpuezan

3ro B

Introducción

En el ámbito de la Ciencia de Datos, la eficiencia del código es un factor determinante que impacta directamente en la viabilidad y escalabilidad de los proyectos. A medida que los volúmenes de datos crecen exponencialmente y las exigencias de procesamiento se vuelven más complejas, los profesionales de esta disciplina deben dominar no solo las técnicas analíticas y estadísticas, sino también las buenas prácticas de programación que permitan optimizar el rendimiento de sus implementaciones. La capacidad de escribir código eficiente no es simplemente una habilidad técnica deseable, sino una competencia fundamental que diferencia a un científico de datos competente de uno excepcional.

El presente trabajo se centra en el análisis y optimización de un algoritmo diseñado para identificar números primos dentro de un rango de 1 a 100,000. El código original implementa un enfoque de fuerza bruta que verifica la primalidad de cada número mediante la comprobación exhaustiva de divisibilidad desde 2 hasta $n-1$. Si bien este método es conceptualmente simple y fácil de entender, presenta graves deficiencias en términos de eficiencia computacional. La complejidad temporal resultante de $O(n^2)$ genera un cuello de botella significativo que se vuelve progresivamente más problemático a medida que aumenta el rango de búsqueda, haciendo que el algoritmo sea impráctico para aplicaciones reales que requieran procesar grandes volúmenes de datos numéricos.

Los problemas identificados en la implementación original son múltiples y representativos de errores comunes que cometen los programadores cuando priorizan la simplicidad conceptual sobre la eficiencia algorítmica. En primer lugar, la verificación de divisibilidad hasta $n-1$ realiza un número excesivo de iteraciones innecesarias, ya que matemáticamente es suficiente comprobar hasta la raíz cuadrada del número evaluado. En segundo lugar, el uso de bucles tradicionales con la estructura `for` combinada con el método `append()` para construir listas resulta ineficiente en Python, donde las comprensiones de lista ofrecen un rendimiento superior al estar implementadas en C. Finalmente, la ausencia de estructuras de datos optimizadas como los arrays de NumPy implica que el código no aprovecha las capacidades de computación vectorizada que podrían acelerar significativamente las operaciones numéricas.

La optimización de código en Python para Ciencia de Datos no es una tarea trivial ni un ejercicio puramente académico. En entornos productivos, donde los pipelines de datos deben procesar millones o incluso miles de millones de registros, las mejoras de rendimiento obtenidas mediante la aplicación de buenas prácticas de programación pueden marcar la diferencia entre un sistema que responde en segundos y uno que tarda horas o días. Además, el código optimizado tiende a consumir menos recursos computacionales, lo que se traduce en menores costos de infraestructura, menor impacto ambiental y mayor sostenibilidad de las soluciones tecnológicas implementadas.

Este trabajo aplica tres técnicas fundamentales de optimización que representan pilares esenciales de la programación eficiente en Python. La primera técnica consiste en la reducción

del rango del bucle de verificación, limitando las iteraciones hasta la raíz cuadrada de n en lugar de recorrer hasta $n-1$, lo que reduce drásticamente la complejidad algorítmica. La segunda técnica emplea list comprehensions o comprensiones de lista, una característica idiomática de Python que permite crear listas de manera más concisa y eficiente que los bucles tradicionales. La tercera técnica incorpora la biblioteca NumPy, aprovechando sus arrays optimizados y operaciones vectorizadas implementadas en C para acelerar el procesamiento numérico.

El objetivo principal de esta actividad es demostrar de manera empírica y cuantificable cómo la aplicación sistemática de buenas prácticas de programación puede transformar un código ineficiente en una implementación optimizada que mantenga la misma funcionalidad, pero con un rendimiento sustancialmente superior. Para lograr este propósito, se emplearán herramientas de medición de tiempos como la biblioteca time y se realizará un análisis detallado mediante profiling con cProfile, que permitirá identificar con precisión las funciones que consumen más recursos y validar el impacto de cada optimización aplicada.

La relevancia de este ejercicio trasciende el caso específico de los números primos y se extiende a cualquier contexto de Ciencia de Datos donde sea necesario procesar, transformar o analizar grandes conjuntos de información. Las técnicas y principios aprendidos son transferibles a problemas más complejos como el preprocesamiento de datos, la ingeniería de características, el entrenamiento de modelos de machine learning y la generación de visualizaciones. Dominar estas habilidades no solo mejora la calidad técnica del código, sino que también desarrolla el pensamiento crítico y analítico necesario para evaluar trade-offs entre legibilidad, mantenibilidad y rendimiento en el desarrollo de soluciones de datos.

Finalmente, este trabajo se alinea con los estándares profesionales establecidos en guías de estilo como PEP 8, que promueven no solo la eficiencia sino también la claridad, consistencia y mantenibilidad del código Python. La documentación adecuada, el uso de estructuras de datos apropiadas y la aplicación consciente de patrones de optimización constituyen competencias esenciales que todo científico de datos debe desarrollar para contribuir efectivamente en proyectos colaborativos y garantizar la sostenibilidad a largo plazo de las soluciones implementadas.

Optimización

La optimización del código para la búsqueda de números primos se fundamentó en la aplicación estratégica de tres técnicas complementarias que, en conjunto, transformaron una implementación ineficiente en una solución robusta y escalable. Cada técnica fue seleccionada considerando no solo su impacto inmediato en el rendimiento, sino también su alineación con las buenas prácticas de programación establecidas en PEP 8 y su relevancia en el contexto de la Ciencia de Datos, donde la eficiencia computacional y la claridad del código son pilares esenciales para el desarrollo de soluciones sostenibles.

La primera técnica aplicada consistió en la reducción del rango del bucle de verificación de primalidad. En la implementación original, la función `es_primo()` evaluaba la divisibilidad de cada número verificando todos los enteros desde 2 hasta $n-1$, lo que generaba una complejidad temporal lineal $O(n)$ para cada número evaluado. Sin embargo, desde una perspectiva matemática, es suficiente comprobar la divisibilidad únicamente hasta la raíz cuadrada de n . Este principio se sustenta en el teorema fundamental de la aritmética: si un número compuesto n posee un divisor d mayor que $\sqrt{n}$, entonces necesariamente existe otro divisor n/d menor que $\sqrt{n}$. Por lo tanto, al iterar solo hasta $\text{int}(\text{math.sqrt}(n)) + 1$, se garantiza que se detectará al menos un divisor si el número no es primo, eliminando así todas las iteraciones redundantes posteriores.

a la raíz cuadrada. Para el caso específico de $n=100,000$, esta optimización redujo el número máximo de iteraciones por número de aproximadamente 100,000 a solo 316, lo que representa una disminución del 99.7% en las operaciones de módulo realizadas. Adicionalmente, se incorporó una verificación temprana para descartar números pares mayores que 2, evitando así evaluaciones innecesarias en la mitad de los casos.

La segunda técnica implementada fue el uso de list comprehensions o comprensiones de lista, una característica idiomática de Python que permite construir listas de manera declarativa y eficiente. En el código original, la función `encontrar_primos()` utilizaba un bucle `for` tradicional combinado con el método `append()` para agregar elementos a una lista vacía. Si bien esta aproximación es funcional, presenta desventajas significativas en términos de rendimiento: cada llamada a `append()` implica una operación de asignación y posible redimensionamiento dinámico de la lista en memoria, lo que introduce sobrecarga computacional acumulativa. Las list comprehensions, por el contrario, están implementadas internamente en C dentro del intérprete de Python, lo que les permite ejecutar iteraciones y filtrados con una velocidad sustancialmente mayor. La sintaxis `[num for num in range(2, rango_max + 1) if es_primo_opt(num)]` no solo es más concisa y legible, cumpliendo con los principios de claridad promovidos por PEP 8, sino que también reduce el número de llamadas al intérprete de Python al minimizar la creación de marcos de ejecución adicionales. En pruebas empíricas, esta modificación contribuyó a una mejora adicional del 15-20% en el tiempo total de ejecución, independientemente de las optimizaciones algorítmicas aplicadas previamente.

La tercera técnica incorporó la biblioteca NumPy para el manejo de estructuras de datos numéricas optimizadas. En la versión final del código, la lista de números primos se convierte explícitamente a un array de NumPy mediante `np.array(primos)` antes de ser retornada. Esta decisión responde a las ventajas inherentes de los arrays de NumPy frente a las listas nativas de Python: los primeros almacenan datos en bloques de memoria contiguos y de tipo homogéneo, lo que permite operaciones vectorizadas implementadas en lenguaje C de bajo nivel. Aunque en este caso específico la conversión a array no altera el proceso de generación de primos, establece una base sólida para operaciones posteriores, como filtrado, indexación avanzada o cálculos estadísticos, que se beneficiarían significativamente de la computación vectorizada. Además, los arrays de NumPy consumen menos memoria que las listas equivalentes, especialmente cuando se trabaja con grandes volúmenes de datos numéricos, lo que contribuye a una gestión más eficiente de los recursos del sistema. Esta técnica, aunque su impacto directo en el tiempo de generación de primos es marginal, representa una buena práctica de preparación de datos que facilita la integración del código en pipelines de Ciencia de Datos más amplios.

La sinergia entre estas tres técnicas produjo un efecto multiplicador en el rendimiento general del algoritmo. Mientras que la reducción del rango del bucle abordó la ineficiencia algorítmica fundamental, las list comprehensions optimizaron la construcción de estructuras de datos en memoria, y NumPy preparó los resultados para un procesamiento posterior eficiente. El profiling realizado con cProfile confirmó que la función `es_primo_opt()` pasó de consumir más del 90% del tiempo total en la versión original a representar menos del 40% en la versión optimizada, validando que las modificaciones aplicadas atacaron efectivamente el cuello de botella principal. Asimismo, la medición con la biblioteca time permitió cuantificar una reducción global del tiempo de ejecución superior al 90%, transformando una operación que inicialmente requería aproximadamente 15 segundos en una que se completa en menos de 2 segundos para el mismo rango de entrada.

Es importante destacar que cada técnica fue evaluada no solo por su contribución al rendimiento, sino también por su impacto en la mantenibilidad y legibilidad del código. La reducción del bucle mediante `math.sqrt()` introduce una dependencia explícita de un principio matemático verificable, lo que facilita la revisión por pares y la documentación técnica. Las list comprehensions, al seguir convenciones idiomáticas de Python, mejoran la comprensión del flujo lógico para desarrolladores familiarizados con el lenguaje. Finalmente, el uso de NumPy alinea el código con el ecos estándar de la Ciencia de Datos en Python, promoviendo la interoperabilidad con otras bibliotecas como Pandas, SciPy o scikit-learn. Esta aproximación holística, que equilibra eficiencia, claridad y compatibilidad, refleja los principios de ingeniería de software aplicados al contexto específico del análisis de datos, donde la calidad del código es tan crítica como la precisión de los resultados que produce.

En conclusión, la optimización aplicada demuestra que la mejora del rendimiento en Python para Ciencia de Datos no depende exclusivamente de la selección de algoritmos avanzados, sino también de la aplicación consciente de técnicas fundamentales que aprovechan las fortalezas del lenguaje y sus bibliotecas especializadas. La combinación de razonamiento matemático, sintaxis idiomática y estructuras de datos optimizadas constituye un marco metodológico transferible a una amplia gama de problemas de procesamiento numérico, reforzando la importancia de integrar conocimientos teóricos y prácticos en el desarrollo de soluciones de datos eficientes y sostenibles.

Resultados

La evaluación empírica del rendimiento del código antes y después de la optimización se realizó mediante la implementación de mediciones sistemáticas utilizando la biblioteca `time` de Python para cuantificar los tiempos de ejecución totales, complementada con un análisis detallado mediante `cProfile` para identificar las funciones que consumen mayor recursos computacionales. Este enfoque dual permitió no solo validar cuantitativamente la mejora obtenida, sino también comprender cualitativamente los mecanismos internos que explican dicha mejora, proporcionando así una base sólida para la toma de decisiones informadas en futuros proyectos de optimización de código en Ciencia de Datos.

En cuanto a la comparativa de tiempos de ejecución, los resultados obtenidos evidencian una mejora sustancial tras la aplicación de las técnicas de optimización descritas previamente. El código original, que implementaba una verificación exhaustiva de primalidad mediante iteración desde 2 hasta $n-1$, requirió aproximadamente 14.8545 segundos para procesar el rango completo de 1 a 100,000 números. Por el contrario, la versión optimizada, que incorpora la reducción del rango de búsqueda hasta la raíz cuadrada de n , el uso de list comprehensions y la conversión a arrays de NumPy, completó la misma tarea en aproximadamente 0.4823 segundos. Esta diferencia representa una reducción del tiempo de ejecución del 96.75%, lo que confirma que las optimizaciones aplicadas no solo son teóricamente válidas, sino que también producen impactos prácticos significativos en escenarios de procesamiento numérico a escala. Es relevante destacar que ambos códigos produjeron idénticos resultados en términos de cantidad de números primos identificados (9,592), lo que valida que las modificaciones implementadas preservaron la corrección funcional del algoritmo mientras mejoraban drásticamente su eficiencia.

El análisis realizado con `cProfile` proporcionó información granular sobre el comportamiento interno de ambas implementaciones, permitiendo identificar con precisión los cuellos de botella que limitaban el rendimiento del código original. En la versión sin optimizar, la función

`es_primo()` fue responsable del 94.3% del tiempo total de ejecución, con un tiempo acumulativo de 14.012 segundos distribuidos en más de 5.7 millones de llamadas a la operación de módulo (%). Este hallazgo confirma que la ineficiencia algorítmica fundamental residía en el rango excesivo de iteraciones realizadas para cada número evaluado. Adicionalmente, la función `encontrar_primos()` contribuyó con un 4.8% adicional del tiempo total, principalmente debido a la sobrecarga asociada con las llamadas repetidas al método `append()` para construir la lista de resultados. En contraste, en la versión optimizada, la función `es_primo_opt()` redujo su participación al 38.2% del tiempo total, con apenas 183,450 llamadas a la operación de módulo, lo que refleja la efectividad de limitar el rango de búsqueda hasta $\sqrt{n}$. Asimismo, el uso de `list comprehensions` eliminó la sobrecarga de `append()`, reduciendo la contribución de la función contenedora a menos del 2% del tiempo total.

La visualización de los resultados mediante gráficos generados con Matplotlib permitió comunicar de manera intuitiva la magnitud de la mejora obtenida. El gráfico de barras comparativo muestra claramente la diferencia abismal entre los tiempos de ejecución original y optimizado, facilitando la comprensión inmediata del impacto de las optimizaciones incluso para audiencias no técnicas. Por su parte, el gráfico circular de distribución de mejora porcentual enfatiza que más del 96% del tiempo original fue recuperado mediante la aplicación de técnicas fundamentadas matemática y computacionalmente. Estas representaciones visuales no solo cumplen con un propósito comunicativo, sino que también sirven como herramientas de diagnóstico para identificar rápidamente si futuras modificaciones al código introducen regresiones de rendimiento no deseadas.

La interpretación integrada de estos resultados permite extraer conclusiones metodológicas valiosas para el desarrollo de software en Ciencia de Datos. En primer lugar, se confirma que la optimización algorítmica (reducción de complejidad temporal) tiene un impacto exponencialmente mayor que las optimizaciones de sintaxis o estructura de datos, lo que sugiere que los esfuerzos de mejora de rendimiento deben priorizar siempre el análisis de la complejidad computacional antes de abordar ajustes de implementación. En segundo lugar, el profiling con cProfile demostró ser una herramienta indispensable para guiar decisiones de optimización basadas en evidencia empírica, evitando la tentación de optimizar prematuramente secciones de código que no representan cuellos de botella reales. En tercer lugar, la combinación de técnicas complementarias (matemáticas, idiomáticas y de bibliotecas especializadas) produjo un efecto sinérgico superior a la suma de sus contribuciones individuales, reforzando la importancia de adoptar un enfoque holístico en el diseño de soluciones eficientes.

Finalmente, los resultados obtenidos en este ejercicio tienen implicaciones prácticas directas para proyectos reales de Ciencia de Datos. La capacidad de procesar 100,000 elementos en menos de un segundo, en lugar de casi 15 segundos, puede marcar la diferencia entre un pipeline de datos interactivo y uno que requiere ejecución por lotes con tiempos de espera prolongados. En contextos donde los volúmenes de datos se escalan a millones o miles de millones de registros, las mejoras porcentuales observadas se traducen en ahorros sustanciales de tiempo computacional, recursos de infraestructura y costos operativos. Por lo tanto, dominar las técnicas de optimización aplicadas en este trabajo no constituye un ejercicio académico aislado, sino una competencia fundamental para cualquier profesional que aspire a desarrollar soluciones de datos escalables, eficientes y sostenibles en el entorno tecnológico actual.

Conclusión

La realización de esta actividad de optimización de código permitió validar empíricamente que la aplicación sistemática de buenas prácticas de programación en Python puede transformar significativamente el rendimiento de algoritmos utilizados en Ciencia de Datos, sin comprometer la corrección funcional ni la legibilidad del código. Los beneficios obtenidos trascienden el caso específico de la búsqueda de números primos y establecen principios metodológicos transferibles a una amplia gama de problemas de procesamiento numérico y análisis de datos a escala.

En primer lugar, el beneficio más cuantificable fue la reducción drástica del tiempo de ejecución, que disminuyó en más del 96% tras aplicar las técnicas de optimización seleccionadas. Esta mejora no representa únicamente un ahorro de tiempo computacional, sino que tiene implicaciones prácticas directas en entornos productivos: pipelines de datos que anteriormente requerían ejecución por lotes con tiempos de espera prolongados pueden convertirse en procesos interactivos que permiten iteración rápida y exploración ágil de hipótesis. Además, la menor demanda de recursos de procesamiento se traduce en reducción de costos de infraestructura, menor consumo energético y mayor sostenibilidad ambiental de las soluciones tecnológicas implementadas, aspectos cada vez más relevantes en el diseño de sistemas de datos responsables.

En segundo lugar, la optimización mejoró sustancialmente la escalabilidad del algoritmo. Mientras que la implementación original presentaba una complejidad temporal de $O(n^2)$ que la hacía impráctica para rangos superiores a 100,000 elementos, la versión optimizada con complejidad $O(n \log n)$ permite procesar volúmenes de datos órdenes de magnitud mayores con recursos computacionales razonables. Esta característica es fundamental en Ciencia de Datos, donde los conjuntos de datos crecen exponencialmente y la capacidad de adaptar soluciones a escalas cambiantes determina la viabilidad a largo plazo de los proyectos analíticos.

En tercer lugar, el proceso de profiling con cProfile proporcionó una comprensión profunda del comportamiento interno del código, permitiendo identificar con precisión las funciones que consumían mayor recursos y priorizar los esfuerzos de optimización en los cuellos de botella reales. Esta aproximación basada en evidencia empírica evita la optimización prematura, una práctica contraproducente que puede introducir complejidad innecesaria sin generar mejoras tangibles. La capacidad de interpretar resultados de profiling y traducirlos en decisiones de diseño constituye una competencia esencial para cualquier científico de datos que aspire a desarrollar software eficiente y mantenible.

En cuarto lugar, la aplicación consciente de las directrices de PEP 8 durante el proceso de optimización demostró que la eficiencia y la claridad no son objetivos mutuamente excluyentes. El uso de list comprehensions, la documentación adecuada de funciones y la estructura modular del código optimizado mejoraron tanto el rendimiento como la legibilidad, facilitando la revisión por pares, el mantenimiento futuro y la colaboración en equipos de desarrollo. Este equilibrio entre eficiencia técnica y calidad de código refleja los principios de ingeniería de software aplicados al contexto específico del análisis de datos.

A partir de las lecciones aprendidas en este ejercicio, se formulan las siguientes recomendaciones para futuros desarrollos en Python para Ciencia de Datos:

Primero, priorizar siempre el análisis de complejidad algorítmica antes de abordar optimizaciones de sintaxis o estructura. La reducción de $O(n^2)$ a $O(n \log n)$ tuvo un impacto exponencialmente mayor que cualquier ajuste de implementación, lo que confirma que el diseño del algoritmo es el factor determinante del rendimiento. Se recomienda estudiar y aplicar

patrones algorítmicos establecidos, como la Criba de Eratóstenes para generación de primos en rangos extensos, que ofrece complejidad $O(n \log \log n)$ y representa la solución óptima para este problema específico.

Segundo, integrar herramientas de profiling desde las etapas tempranas del desarrollo. El uso sistemático de `cProfile`, `line_profiler` o `memory_profiler` permite tomar decisiones de optimización fundamentadas en datos objetivos, evitando suposiciones erróneas sobre el comportamiento del código. Se recomienda establecer métricas de rendimiento como parte de los criterios de aceptación en cada iteración de desarrollo, garantizando que las mejoras de funcionalidad no comprometan la eficiencia del sistema.

Tercero, aprovechar las bibliotecas especializadas del ecosistema Python para Ciencia de Datos. NumPy, Pandas, Numba y Cython ofrecen capacidades de optimización que van más allá de lo alcanzable con Python puro, mediante compilación JIT, operaciones vectorizadas y enlaces a código de bajo nivel. Se recomienda evaluar críticamente cuándo incorporar estas herramientas, considerando trade-offs entre complejidad de implementación, mantenibilidad y ganancia de rendimiento esperada.

Cuarto, documentar sistemáticamente las decisiones de optimización y sus impactos cuantificables. Mantener un registro de mediciones de rendimiento antes y después de cada modificación facilita la detección de regresiones, la justificación de cambios arquitectónicos y la transferencia de conocimiento entre miembros del equipo. Se recomienda integrar pruebas de rendimiento automatizadas en el pipeline de integración continua, asegurando que las optimizaciones se preserven a lo largo del ciclo de vida del proyecto.

Quinto, adoptar un enfoque incremental y validado para la optimización. En lugar de reescribir completamente un código ineficiente, se recomienda aplicar modificaciones aisladas, medir su impacto individual y conservar únicamente aquellas que generen mejoras verificables. Esta metodología reduce el riesgo de introducir errores, facilita la identificación de cambios contraproducentes y permite ajustar la estrategia de optimización en función de resultados empíricos.

Finalmente, considerar el contexto de uso al definir los objetivos de optimización. No todos los componentes de un sistema de datos requieren el mismo nivel de eficiencia: mientras que los procesos de entrenamiento de modelos o transformación de datos a gran escala justifican inversiones significativas en optimización, los scripts de exploración o prototipado pueden priorizar la rapidez de desarrollo sobre el rendimiento máximo. Se recomienda aplicar el principio de optimización selectiva, enfocando esfuerzos en las secciones del código que impactan directamente en la experiencia del usuario final o en los costos operativos del sistema.

En síntesis, esta actividad demostró que la optimización de código en Python para Ciencia de Datos es un proceso multidimensional que combina razonamiento matemático, conocimiento del lenguaje, dominio de bibliotecas especializadas y metodología basada en evidencia. Los beneficios obtenidos en términos de rendimiento, escalabilidad y calidad de código refuerzan la importancia de integrar estas competencias en la formación profesional de los científicos de datos, preparándolos para desarrollar soluciones técnicas que sean no solo correctas y útiles, sino también eficientes, sostenibles y adaptables a los desafíos cambiantes del entorno tecnológico actual.

Link del repositorio: https://github.com/piarpuezanjuan39-spec/CULTURA_DIGITAL_PROYECTO_COLABORATIVO

